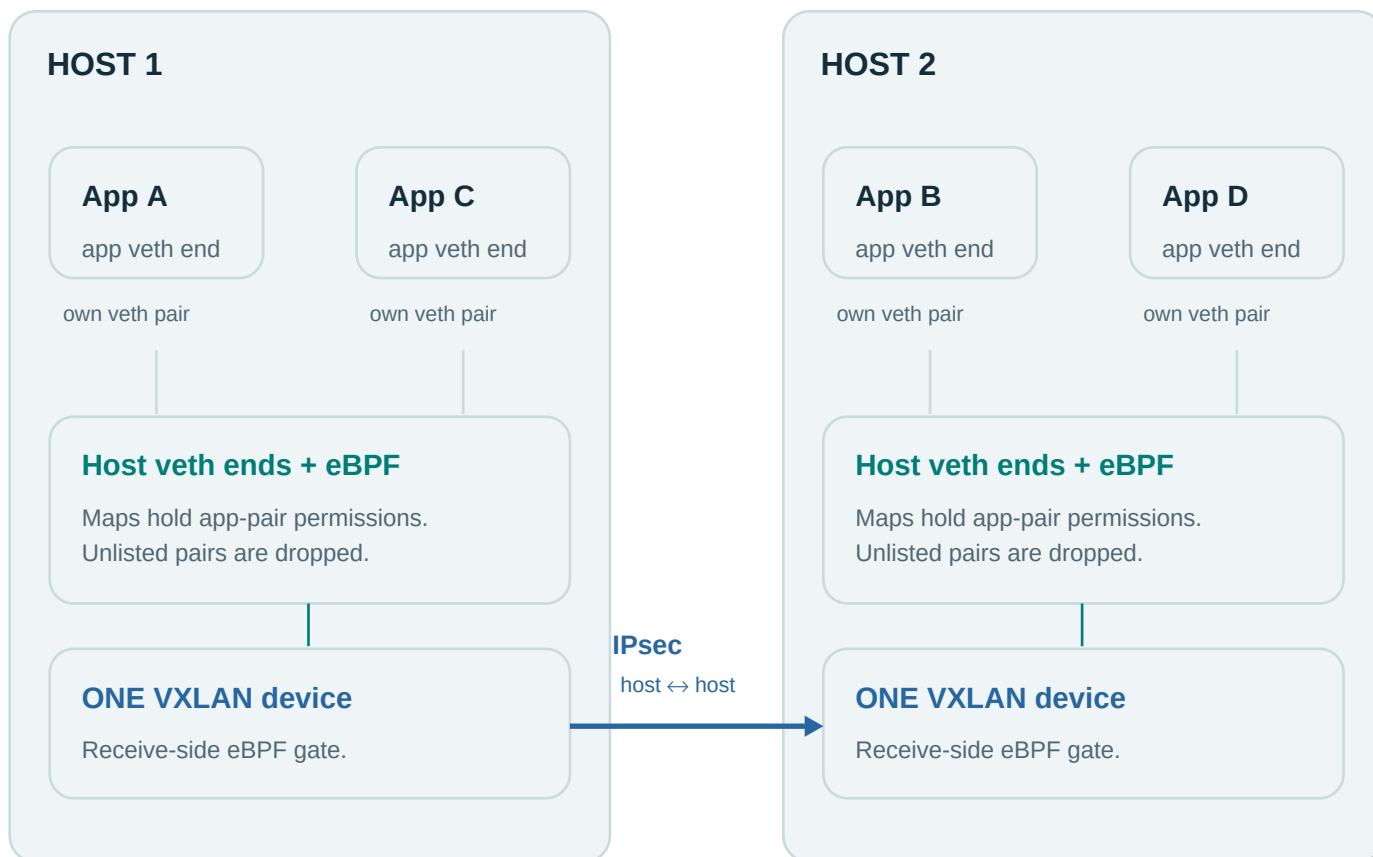


One transport. Separate app permissions.

VXLAN is shared by applications on the hosts; it is not created per app pair.

Example: allow App A to talk to App B



App A → B: passes both hosts' permission checks.

App C → D: dropped. A shared VXLAN is not a shared app LAN.

No Linux bridge: eBPF redirects packets between the veth and VXLAN interfaces.

What is shared?

One VXLAN device per host; one encrypted transport per host pair.

Each app namespace has its own attachment and separate pair permissions.

Attachment = a veth pair (virtual Ethernet cable).

App end: an interface in the app's namespace, with a fixed local IP and route.

Host end: a separate interface where eBPF checks the app's packets.

It connects the app to the gate; it grants no permission by itself.

For the proxy, the app end is in the host's own network namespace.

Created once, reused

Attach the app when first needed. Opening / closing an app-pair connection adds / removes permissions; the attachment and host transport stay in place.

First integration: opt-in, encrypted cross-host paths. Other paths retain the existing backend.

A permission is a packet-checking rule.

Nullnet installs these rules in eBPF maps on both hosts.

Example permission: App A ↔ App B, generation 42

This grants connectivity between two app namespaces; it is not a per-port firewall rule.

What the rule contains

Local app attachment + local and remote connection IP addresses.
Expected remote host + authenticated encrypted transport.
Generation number + send-enabled state. A host-wide deadline must also be valid.

What happens to a packet?

1 Sending host

Find rule for this attachment + peer.
Check enabled state and deadline.
Translate app IP to connection IP.
Add generation; send via encrypted VXLAN.



2 Receiving host

Check authenticated host + app addresses.
Match generation; check deadline.
Translate to the app's local IP.
Deliver only to the named app attachment.

No matching rule, stale generation or expired deadline → drop.

Open, use, close

Prepare → activate

Install receive rules on both hosts while sends remain blocked.
After both hosts acknowledge preparation, enable sends.

Revoke

Delete the pair's rules on both hosts. Further packets are dropped.
Keep the shared devices and other app pairs' permissions.

Reuse safely

Use a new generation when a connection slot is reused.
Old-generation packets fail the checks; socket reuse also needs explicit fencing.

These are host-enforced rules, not credentials given to applications.

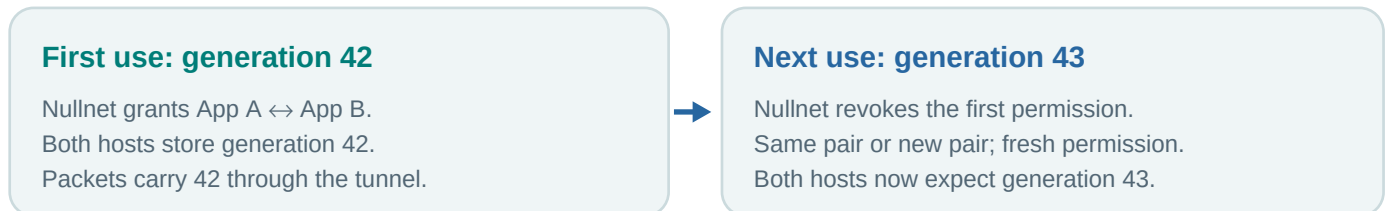
Control heartbeats renew the shared deadline; loss of renewal blocks traffic.

Revocation stops further traffic; it does not close application sockets or recall admitted packets.

A generation identifies one use of a slot.

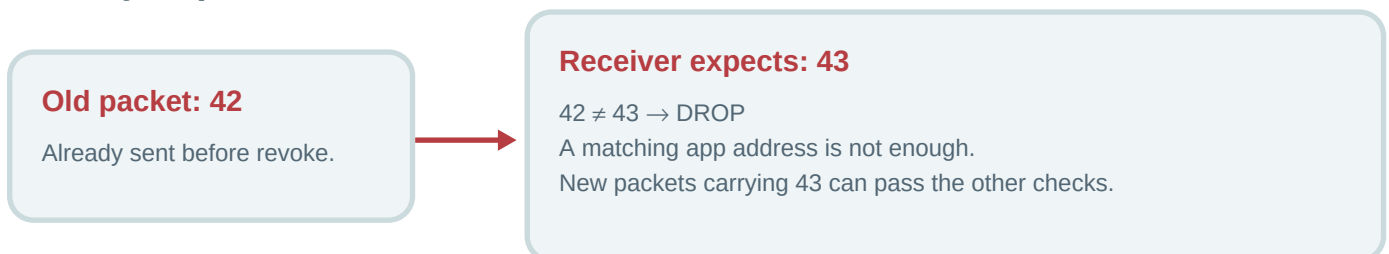
The connection address may be reused. Its generation must change.

Example: the same connection slot, used twice



The number is assigned by the controller, not the application.

A delayed packet arrives after reuse



How the check works

1. On allocation, the controller assigns a fresh, increasing number.
2. It sends the same number to both hosts over the TLS control channel.
3. The sender's eBPF rule writes that number into VXLAN metadata.
4. IPsec protects the packet and its generation in transit.
5. The receiver requires an exact match with its current permission.

A generation is not a password or a timeout.

It separates successive grants that reuse an address or connection slot.

IPsec authenticates the remote host; the heartbeat deadline controls expiry.

Why TCP needs another check

An old socket can emit a new packet after reuse; the sender would label it 43.
The integration therefore records the generation when a TCP SYN opens a flow.
Later TCP packets must match that flow generation; an old established flow stays blocked.

Integration encoding: 40 bits = 24-bit VXLAN VNI + 16-bit GBP policy ID; do not wrap.

Restart boundary: fresh transport keys distinguish controller / host incarnations.

Hundreds of full cycles per second.

Measured in Nullnet on two Linux hosts with eight test containers each.

Complete connection cycles per second

Encrypted HTTP connection setup + traffic + completed retirement

Dedicated pooled backend

69

Median, $3 \times 1,024$

Policy backend

743

Median, $3 \times 1,024$

Policy, sustained load

$\geq 1,716$

60 s + final drain

107,696 complete cycles in the sustained run. Zero failures.

Two Linux hosts; eight test containers per host; concurrency 64.

The sustained run reuses infrastructure; every counted connection is new.

Why the hot path is smaller

Dedicated topology

Create / configure / delete devices



Policy lifecycle

Update / delete permission entries

Control-plane map work: 6 updates + 4 deletions per two-host cycle.

Host mappings and control acknowledgements still run for each connection.

What the integration preserves

Bind identity to the application attachment; authenticate the remote host.

Reject old generations; expire permissions when renewal stops.

Retain dedicated paths for same-host networks, backend triggers and egress.

Define socket reuse: revocation blocks traffic but does not close sockets.

Product checks: isolation, generation / TCP fencing, heartbeat expiry,
mixed-topology drain and restart recovery passed. Experimental and opt-in.

Evidence and current status: docs/reports/policy-integration-2026-09-29/README.md